

# **Sri Lanka Institute of Information Technology**



**Data Warehousing and Business Intelligence -**

**IT3021 Assignment 1**

IT23556348 – Wickramanayaka D S

## Table of Contents

|      |                                                |    |
|------|------------------------------------------------|----|
| 1.   | Declaration.....                               | 3  |
| 2.   | Dataset Selection.....                         | 3  |
| 3.   | Preparation of Data Sources.....               | 4  |
| 3.1. | Entity Relationship Diagram.....               | 6  |
| 4.   | High Level Design Architecture .....           | 7  |
| 4.1. | Data Storage Snapshots of SSMS .....           | 8  |
| 5.   | Relational Schema .....                        | 9  |
| 6.   | ETL Development.....                           | 10 |
| 6.1. | Data Extraction and Staging Area Loading ..... | 10 |
| 6.2. | Data Extraction and Warehouse Loading .....    | 13 |

# 1. Declaration

I declare that this assignment report is based on my own work carried out for the Data Warehousing and Business Intelligence module. The design, implementation, ETL processes, database structures, and analysis included in this report were developed by me using the selected dataset and supporting tools. Any external references used in this report are acknowledged appropriately.

## 2. Dataset Selection

Dataset Name : Predicting Coupon Redemption Feature Selection

Provided by : Kaggle.com

Source Link : [Predicting Coupon Redemption Feature selection](#)

This is a data set provided by an established Brick and Mortar retailer that frequently conducts marketing campaign for its diverse product range.

Discount marketing and Coupon usage are very widely used promotional techniques to attract new customers and to retain & enforce loyalty of existing customers.

The data set has information on customer transactions on products and the usage of coupons provided by various campaigns to complete their transactions. The features of this data set allow visualization of data set in various perspectives such as category based, brand based and Item based too. The original data set contains nine csv files with information on Customers , Campaigns , Products in the retailer, Transactions done by customers and the rest are all summary and compilation files of the above mentioned four subject areas.

### 3. Preparation of Data Sources

The entire data set contains various subject content in csv format. In preparation of data sources, some changes were done to create a completed data source with satisfied project criteria. In such aspect few additional details such as customer addresses and contact details were added additionally to the data set. Some changes were made to the format of the source data to convert them into text files and to convert the csv files into a source database.

Final source data used for transformation process are as follows :-

- CouponCustomerAddress.xls – Customer Address Information.
- CouponItemCategory.txt – Category of Items Information in retailer.
- Coupon\_SourceDB
  - dbo.CampaignData
  - dbo.CouponRedemp
  - dbo.customer\_Info
  - dbo.Customer\_ItemTransaction
  - dbo.Items
  - dbo.productBrand

According to the final data set, data related to customers involved in transactions with the company is recorded in dbo.customer\_Info table. The location address of the customer is brought as a separate source namely CouponCustomerAddress.xls. The various campaigns initiated for this marketing process is recorded in dbo.CampaignData table. Coupons offered by campaigns for customer transactions are stored in dbo.CouponRedemp table. The products sold in the Retailer is divided into various categories and is stored in CouponItemCategory.txt, whereas each of these categories can further contain various brands and is stored in dbo.productBrand table. Ultimately each brand can have various items to be sold to the customer and is stored in dbo.Items table, respectively. The customer transactions with the retailer shop are stored in dbo.Customer\_ItemTransaction.

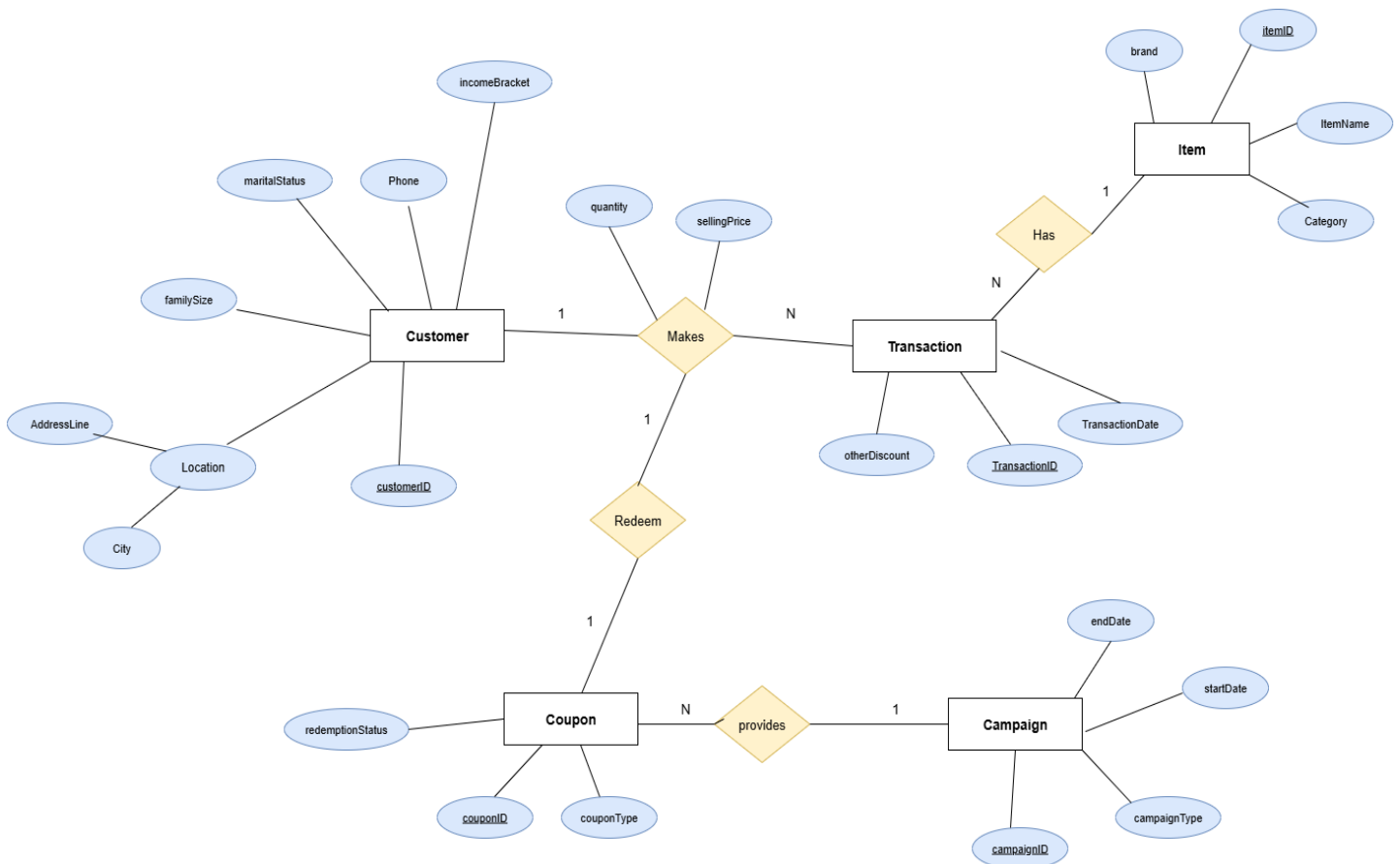
These sources were selected because they collectively represent the core business processes of a retail system. The dataset includes both descriptive data (such as customers, products, and campaigns) and transactional data (such as purchases and coupon redemption), which are essential for designing a dimensional model and supporting analytical processing in the data warehouse.

## Data Descriptions

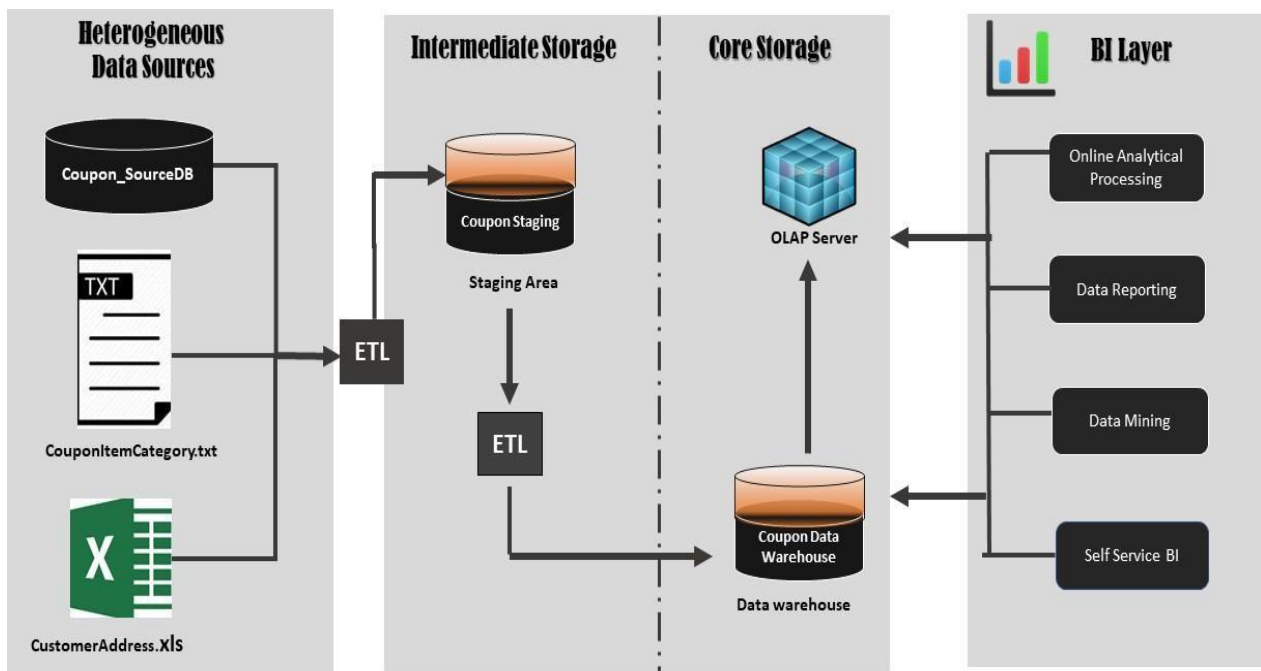
| Source Type | Table Name                 | Data              |               |
|-------------|----------------------------|-------------------|---------------|
| Text file   | CouponItemCategory.txt     |                   |               |
|             |                            | Column Name       | Data type     |
|             |                            | category_id       | NVARCHAR(255) |
|             |                            | category_name     | NVARCHAR(255) |
| Xlsx file   | CouponCustomerAddress.xlsx |                   |               |
|             |                            | Column Name       | Data type     |
|             |                            | customer_id       | NVARCHAR(255) |
|             |                            | address_line      | NVARCHAR(255) |
|             |                            | city              | NVARCHAR(255) |
|             |                            | contact_number    | NVARCHAR(255) |
| Csv file    | CampaignData.csv           |                   |               |
|             |                            | Column Name       | Data type     |
|             |                            | campaign_id       | INT           |
|             |                            | campaign_type     | VARCHAR(10)   |
|             |                            | start_date        | DATE          |
|             |                            | end_date          | DATE          |
| Csv file    | CouponRedemp.csv           |                   |               |
|             |                            | Column Name       | Data type     |
|             |                            | redemption_id     | INT           |
|             |                            | customer_id       | INT           |
|             |                            | campaign_id       | INT           |
|             |                            | coupon_id         | INT           |
|             |                            | redemption_status | INT           |
| Csv file    | customer_Info.csv          |                   |               |
|             |                            | Column Name       | Data type     |
|             |                            | customer_id       | INT           |
|             |                            | age_range         | VARCHAR(20)   |
|             |                            | marital_status    | VARCHAR(20)   |
|             |                            | family_size       | INT           |
|             |                            | income_bracket    | INT           |
| Csv file    | productBrand.csv           |                   |               |
|             |                            | Column Name       | Data type     |
|             |                            | brand_id          | INT           |
|             |                            | brand_type        | VARCHAR(50)   |

| Source Type    | Table Name                    | Data                                                                                                                                                                                                                                                                                                                                                                           |             |           |                |     |          |      |             |     |         |     |          |     |               |                |                |                |
|----------------|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|-----------|----------------|-----|----------|------|-------------|-----|---------|-----|----------|-----|---------------|----------------|----------------|----------------|
| Csv file       | Items.csv                     | <table><tr><th>Column Name</th><th>Data type</th></tr><tr><td>item_id</td><td>INT</td></tr><tr><td>brand_id</td><td>INT</td></tr></table>                                                                                                                                                                                                                                      | Column Name | Data type | item_id        | INT | brand_id | INT  |             |     |         |     |          |     |               |                |                |                |
| Column Name    | Data type                     |                                                                                                                                                                                                                                                                                                                                                                                |             |           |                |     |          |      |             |     |         |     |          |     |               |                |                |                |
| item_id        | INT                           |                                                                                                                                                                                                                                                                                                                                                                                |             |           |                |     |          |      |             |     |         |     |          |     |               |                |                |                |
| brand_id       | INT                           |                                                                                                                                                                                                                                                                                                                                                                                |             |           |                |     |          |      |             |     |         |     |          |     |               |                |                |                |
| Csv file       | Customer_ItemTransaction .csv | <table><tr><th>Column Name</th><th>Data type</th></tr><tr><td>transaction_id</td><td>INT</td></tr><tr><td>date</td><td>DATE</td></tr><tr><td>customer_id</td><td>INT</td></tr><tr><td>item_id</td><td>INT</td></tr><tr><td>quantity</td><td>INT</td></tr><tr><td>selling_price</td><td>DECIMAL(18, 2)</td></tr><tr><td>other_discount</td><td>DECIMAL(18, 2)</td></tr></table> | Column Name | Data type | transaction_id | INT | date     | DATE | customer_id | INT | item_id | INT | quantity | INT | selling_price | DECIMAL(18, 2) | other_discount | DECIMAL(18, 2) |
| Column Name    | Data type                     |                                                                                                                                                                                                                                                                                                                                                                                |             |           |                |     |          |      |             |     |         |     |          |     |               |                |                |                |
| transaction_id | INT                           |                                                                                                                                                                                                                                                                                                                                                                                |             |           |                |     |          |      |             |     |         |     |          |     |               |                |                |                |
| date           | DATE                          |                                                                                                                                                                                                                                                                                                                                                                                |             |           |                |     |          |      |             |     |         |     |          |     |               |                |                |                |
| customer_id    | INT                           |                                                                                                                                                                                                                                                                                                                                                                                |             |           |                |     |          |      |             |     |         |     |          |     |               |                |                |                |
| item_id        | INT                           |                                                                                                                                                                                                                                                                                                                                                                                |             |           |                |     |          |      |             |     |         |     |          |     |               |                |                |                |
| quantity       | INT                           |                                                                                                                                                                                                                                                                                                                                                                                |             |           |                |     |          |      |             |     |         |     |          |     |               |                |                |                |
| selling_price  | DECIMAL(18, 2)                |                                                                                                                                                                                                                                                                                                                                                                                |             |           |                |     |          |      |             |     |         |     |          |     |               |                |                |                |
| other_discount | DECIMAL(18, 2)                |                                                                                                                                                                                                                                                                                                                                                                                |             |           |                |     |          |      |             |     |         |     |          |     |               |                |                |                |

### 3.1. Entity Relationship Diagram



## 4. High Level Design Architecture



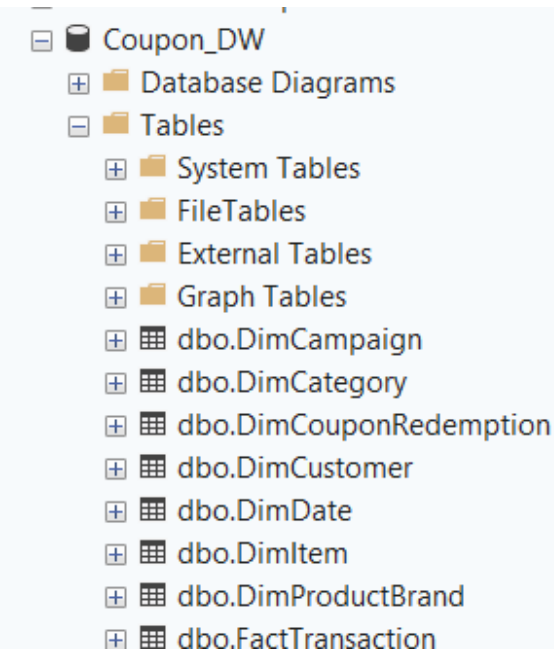
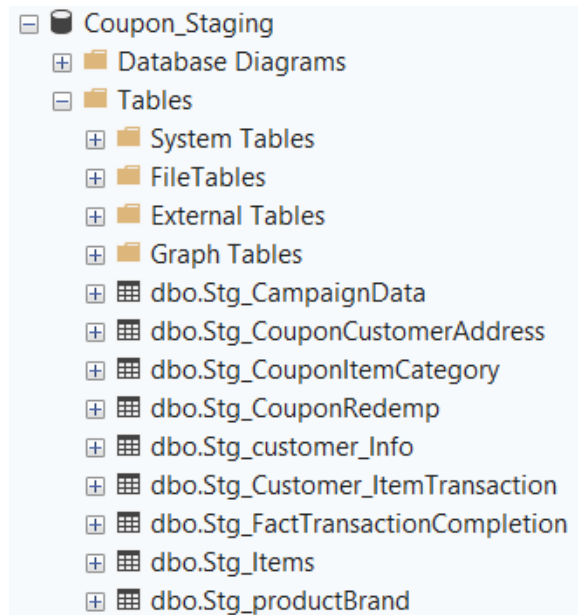
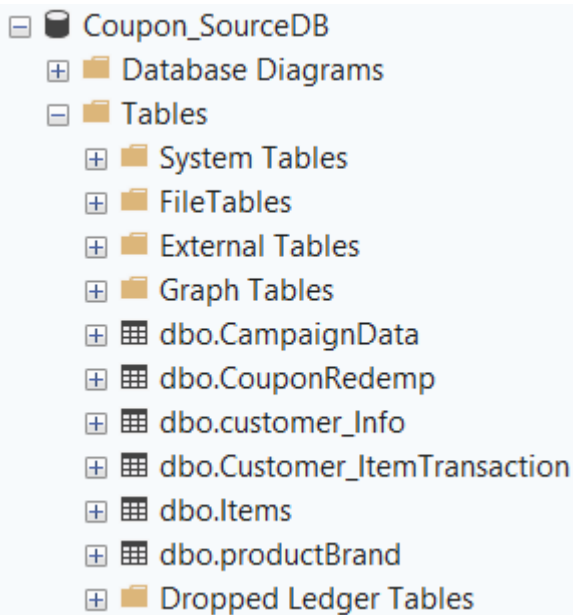
The high-level architecture solution of the Brick-and-Mortar retailer company is as given above. The entire company data is provided by three separate source formats: Source database, Text file and Excel file, respectively.

As per the requirements since we do not need any real time data capturing it was decided to load the entire source data content into an intermediate staging area. Additionally any complex queries performed will not affect the source data performance by doing so. Therefore as the initial step entire source data (Coupon\_SourceDB , CustomerAddress.xls, Category.txt) information are loaded into another schema(Coupon\_Staging) ,the staging database which ultimately serves as a single database containing all the data from various sources compiled at one place.

As the next step necessary transformations are done at the staging layer in order to load data from staging area to data warehouse(Coupon\_DW).

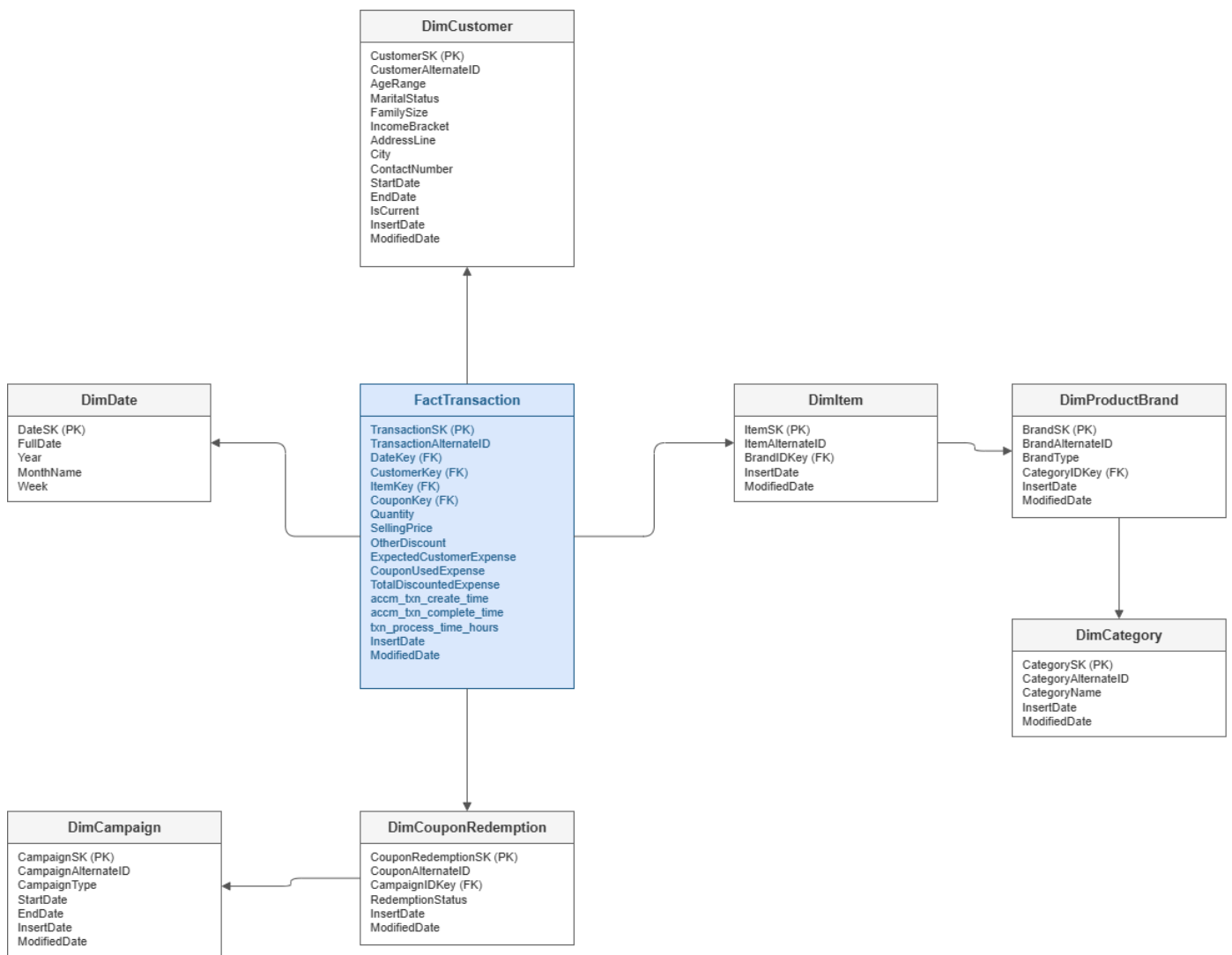
The data from data warehouse is then refreshed to create OLAP cubes which can be used by end users in order to carry out Analysis on the data set. Here data visualization can be either done through OLAP server or directly from Data warehouse which is specifically called Self service BI.

## 4.1. Data Storage Snapshots of SSMS





## 5. Relational Schema



The schema designed for the Brick-and-Mortar retailer follows a **snowflake schema** consisting of one fact table and multiple dimension tables including the date dimension.

Each dimension table is uniquely identified using a **surrogate key**, while also maintaining business keys from the source system. Referential integrity between tables is maintained using **foreign key (FK)** relationships.

**Hierarchical relationships** exist within the schema:

1. DimProductBrand and DimCategory are hierarchical dimensions of DimItem.
2. DimCustomer includes address-based hierarchy attributes such as city and address details.

DimCustomer is implemented as a **Slowly Changing Dimension (SCD)** using Type 2 methodology.

The grain of the FactTransaction table is defined as a single transaction performed by a customer for a specific item on a given date.

Additional **derived calculations** are implemented in the FactTransaction table.

1. ExpectedCustomerExpense = [Quantity] \* [SellingPrice]
2. CouponUsedExpense: = OtherDiscount
3. TotalDiscountedExpense = (Quantity \* SellingPrice) - CouponUsedExpense

The FactTransaction table was extended to support an **accumulating fact table design**.

Three additional columns were added to track the lifecycle of each transaction:

1. accm\_txn\_create\_time – Current system time when ETL runs
2. accm\_txn\_complete\_time – You load this into staging and then update Fact table
3. txn\_process\_time\_hours – represents the duration of the transaction in hours, calculated as the difference between completion time and creation time.  
(Time Difference = Completion Time - Create Time)

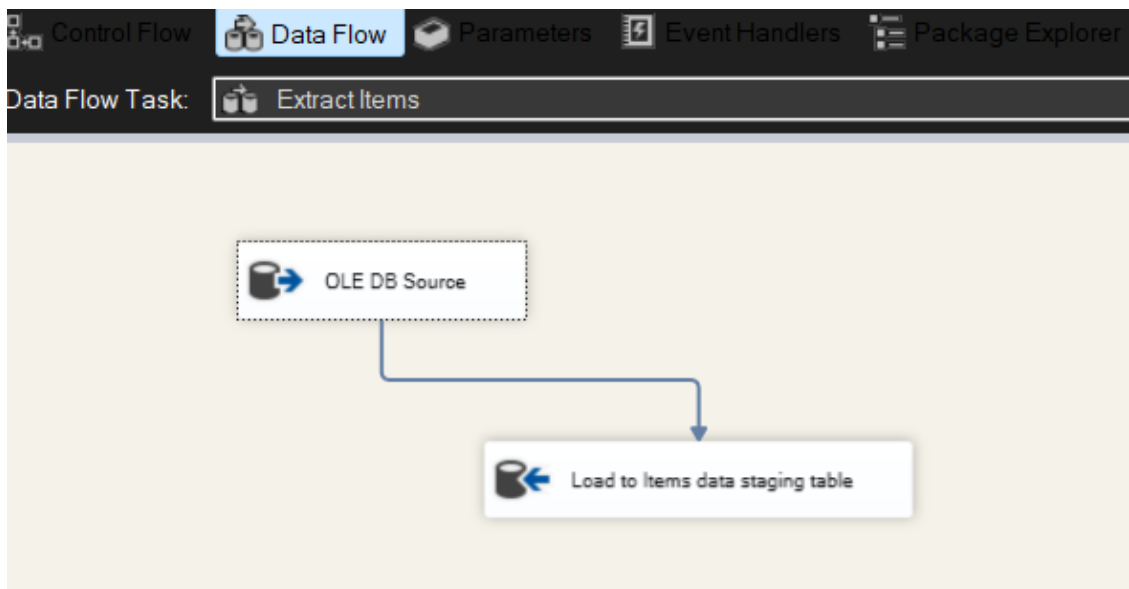
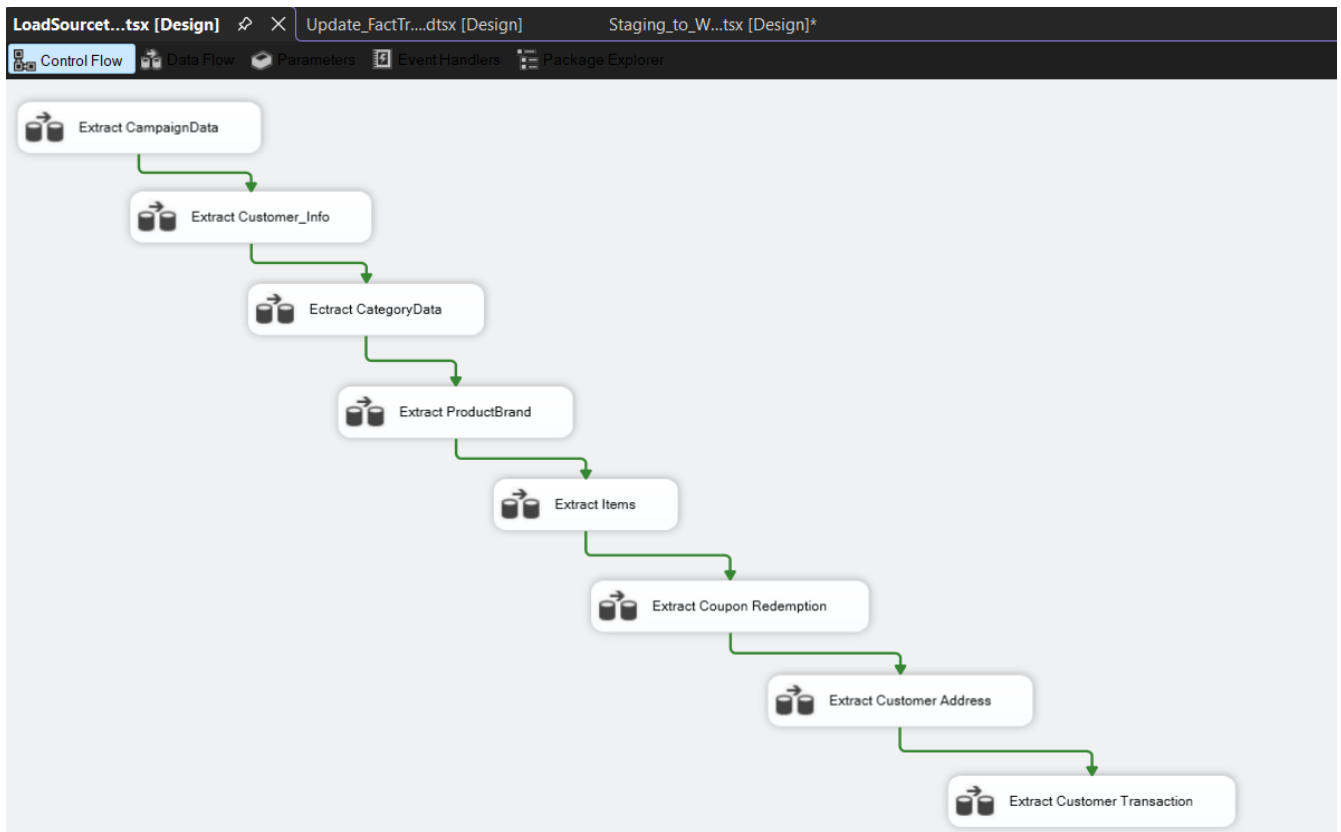
These columns enable tracking of transaction progress over time, which is a key characteristic of accumulating fact tables.

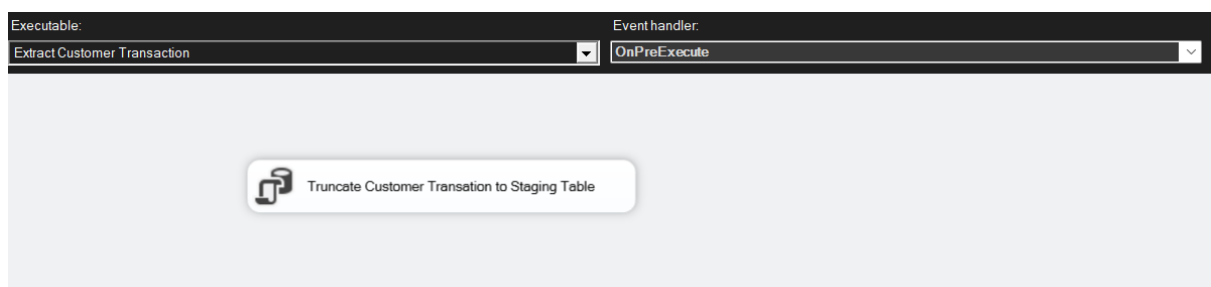
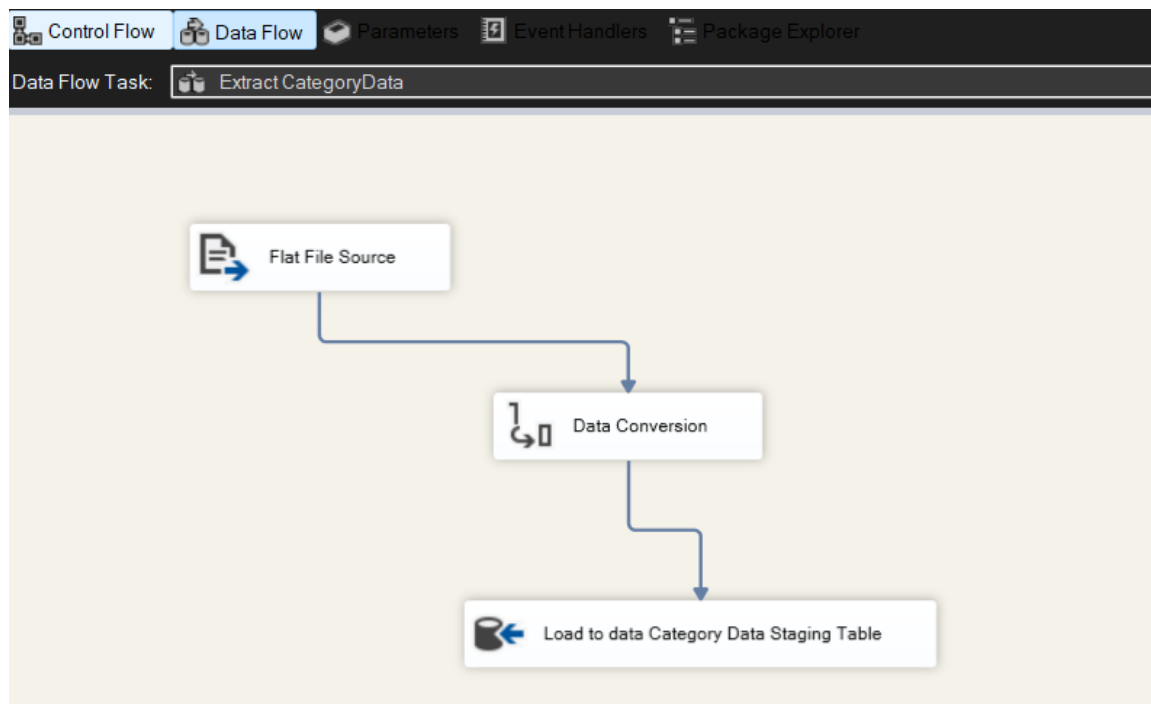
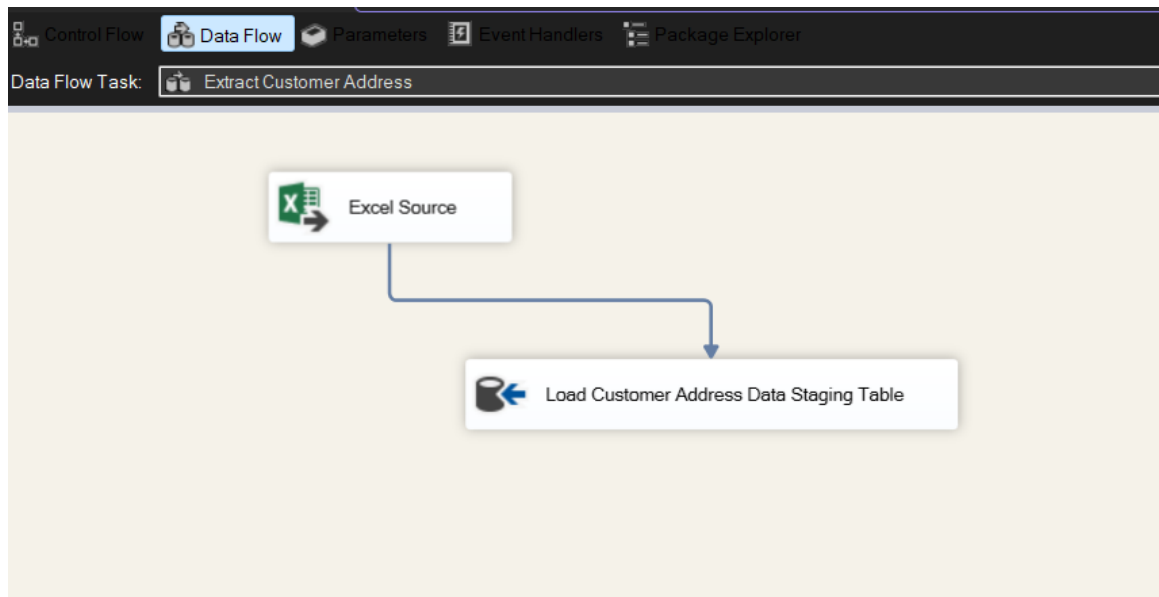
## 6. ETL Development

### 6.1. Data Extraction and Staging Area Loading

Data extraction from source data to staging area is done using SQL Server Data tool 2022 environment. The Source Database, Text file and Excel file created are used in this process. To extract data from Source Database tables, **OLE DB Source** is used in the data flow whereas text file extraction is done using **Flat file source** and Excel file extraction using **Excel Source**. From Source database to staging database lesser transformations are done, primary concern is on Data extraction and Data loading. The extracted Source data are finally loaded into the respective staging tables using **OLE DB Destination**.

Below attached images are some illustrations of different source data Extraction and Loading Process



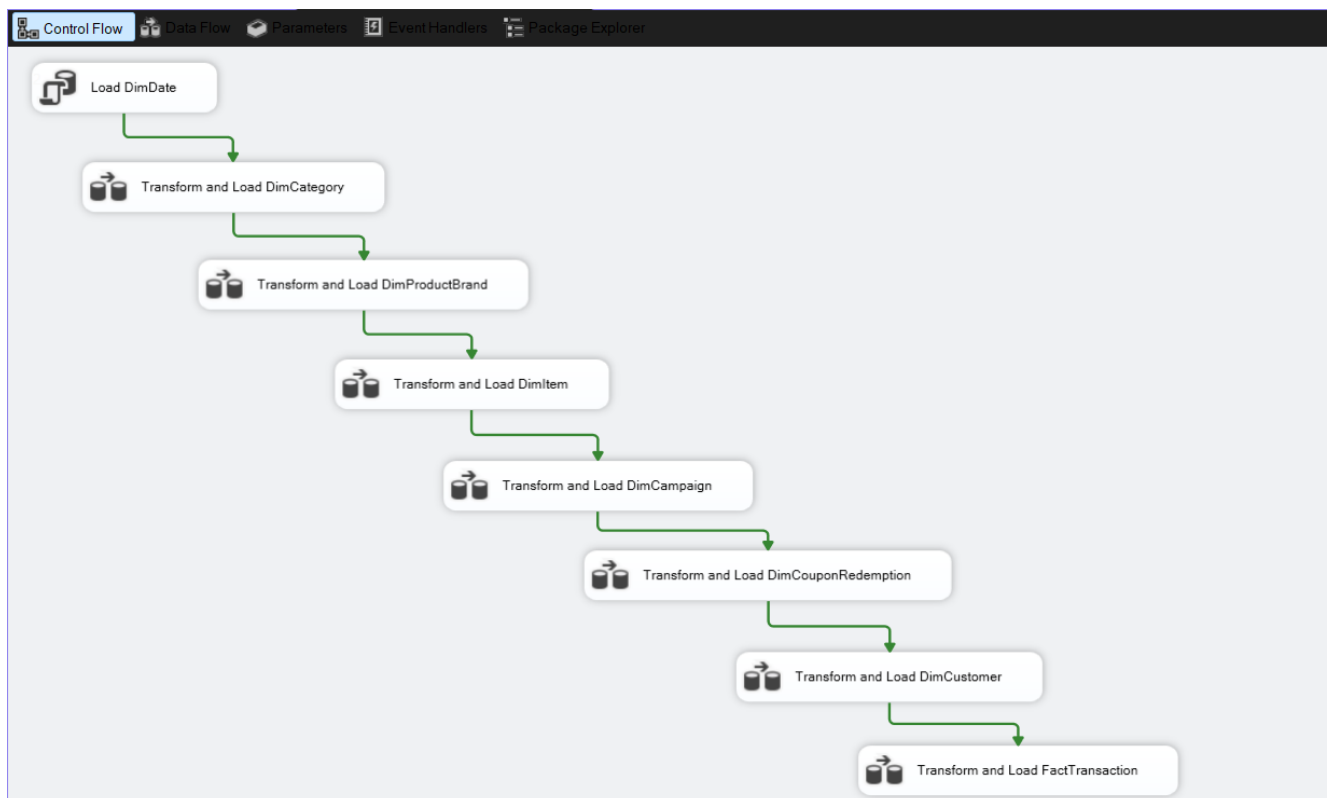


The above order is followed in completing the orchestration of loading data to staging database. Finally, to ensure that data warehouse loading is followed only after loading data to staging database **Execute Package Task** is Used.

To prevent data duplication when staging tables are repeatedly loaded, **On Pre execute event handlers** are used to truncate data before each staging table data loading is initiated.

## 6.2. Data Extraction and Warehouse Loading

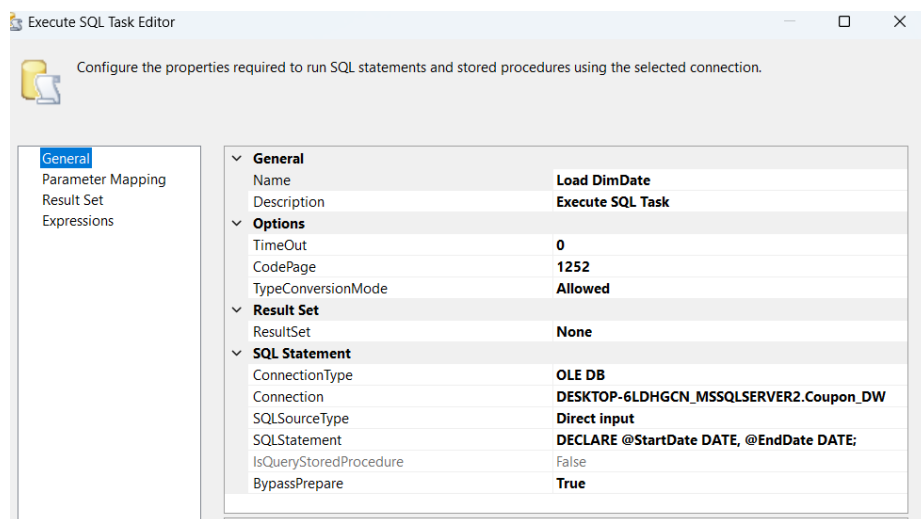
During the data loading process to datawarehouse, the main concern was on the order of execution of tasks. The schema was very carefully analysed in order to identify the dependencies among various tables thus deciding the order of table load executions



## Load DimDate Task

The DimDate table is loaded using a dedicated control flow task. This task generates and inserts date records into the dimension table before loading other dimensions and the fact table.

This ensures that all date-related foreign key references in the fact table can be resolved during the ETL process.



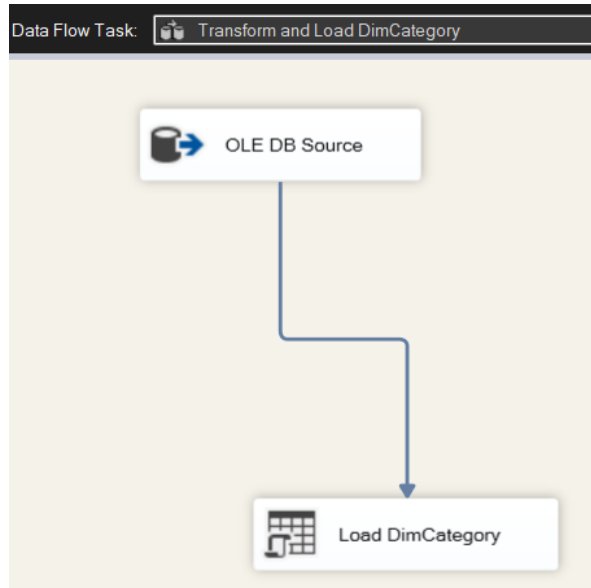
### 6.2.1. Loading Hierarchical Dimensions

Hierarchical dimensions in the data warehouse were loaded following a dependency-based approach to maintain referential integrity.

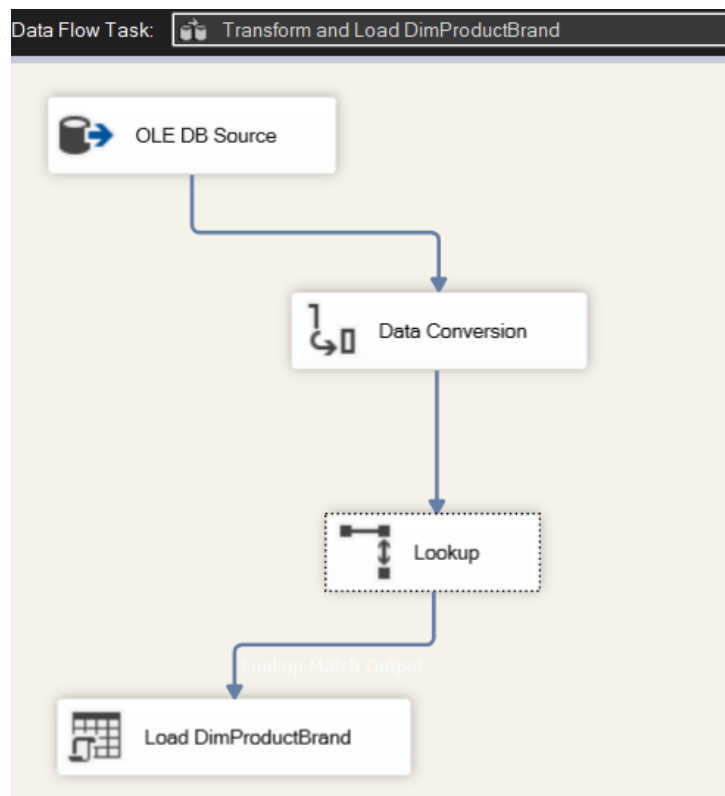
The hierarchy implemented in this project follows a snowflake structure where Category is the highest level, followed by ProductBrand, and then Item.

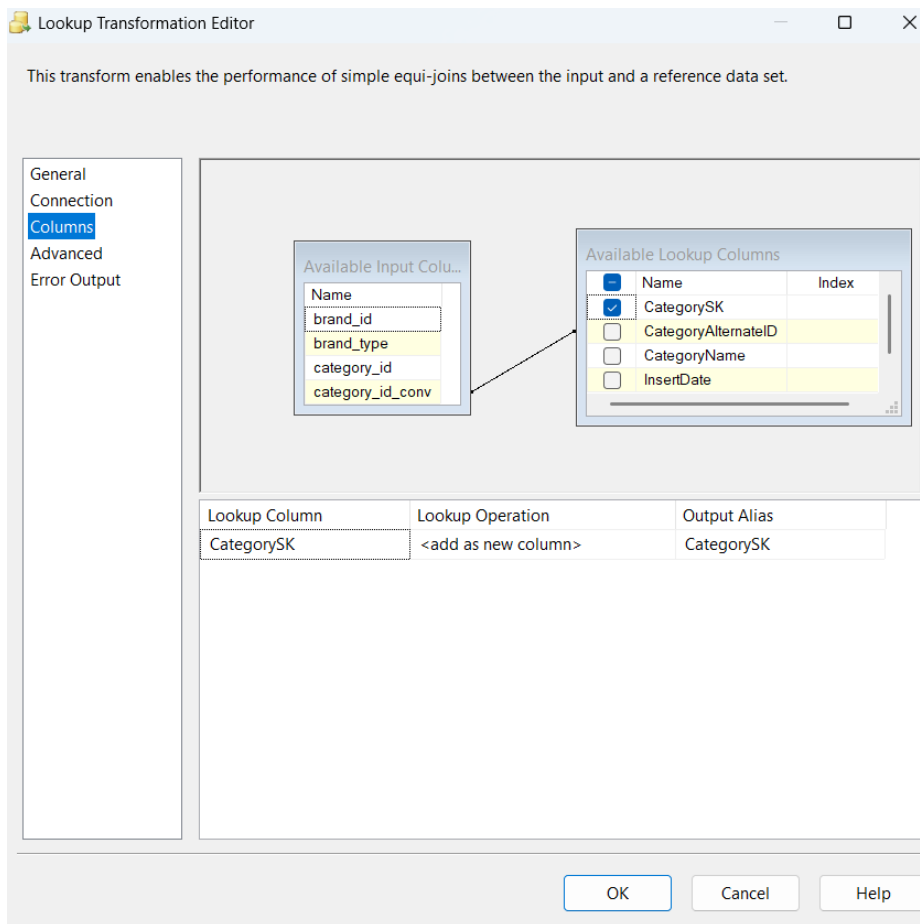
The loading process was performed in the following order:

1. DimCategory was loaded first as it represents the top-level dimension and does not depend on any other table. Data was extracted from the staging area and directly inserted into the dimension table.

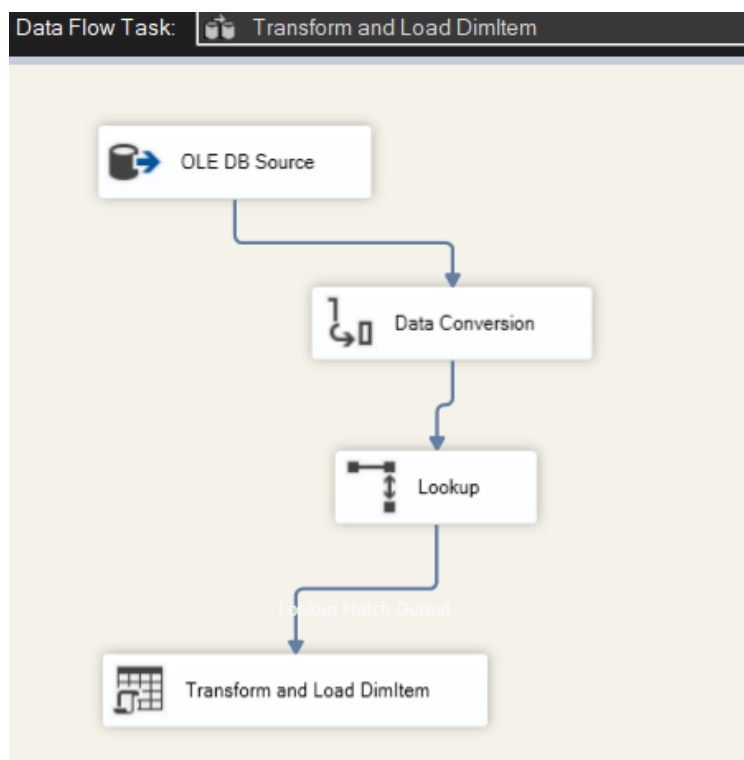


2. DimProductBrand was loaded next. Since it depends on the Category dimension, a Lookup transformation was used to retrieve the corresponding Category surrogate key. Data type conversions were handled using a Data Conversion transformation before loading.

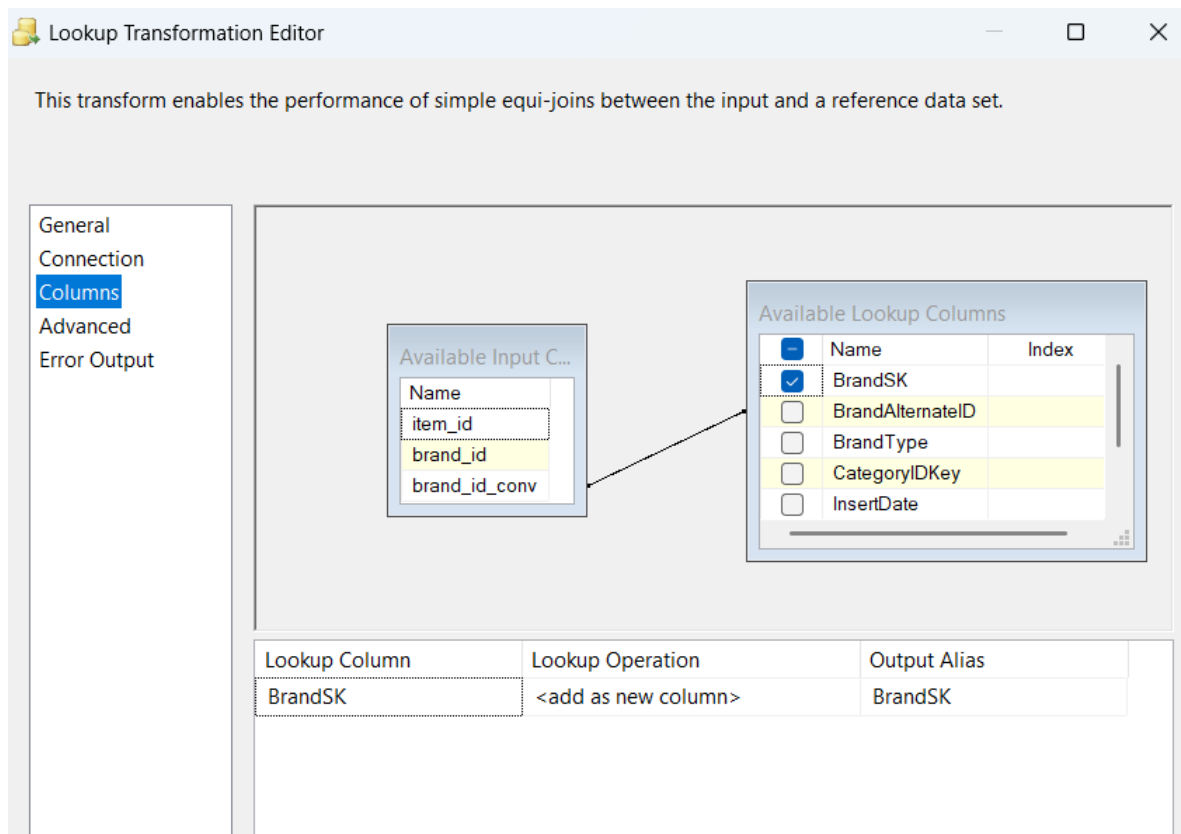




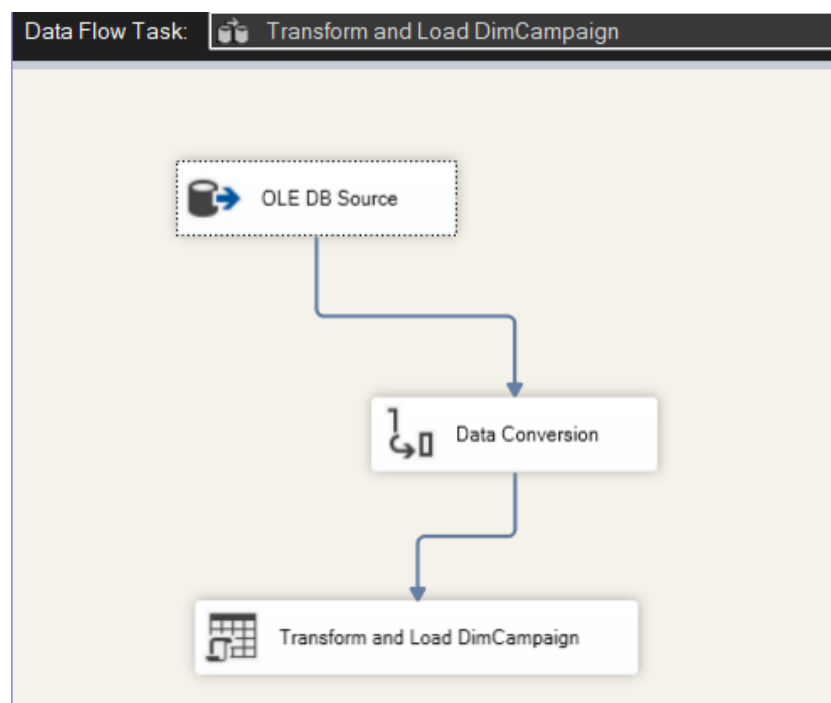
- DimItem was loaded last in the hierarchy. This dimension depends on the ProductBrand table. A Lookup transformation was used to map the appropriate Brand surrogate key before inserting records into the dimension table.

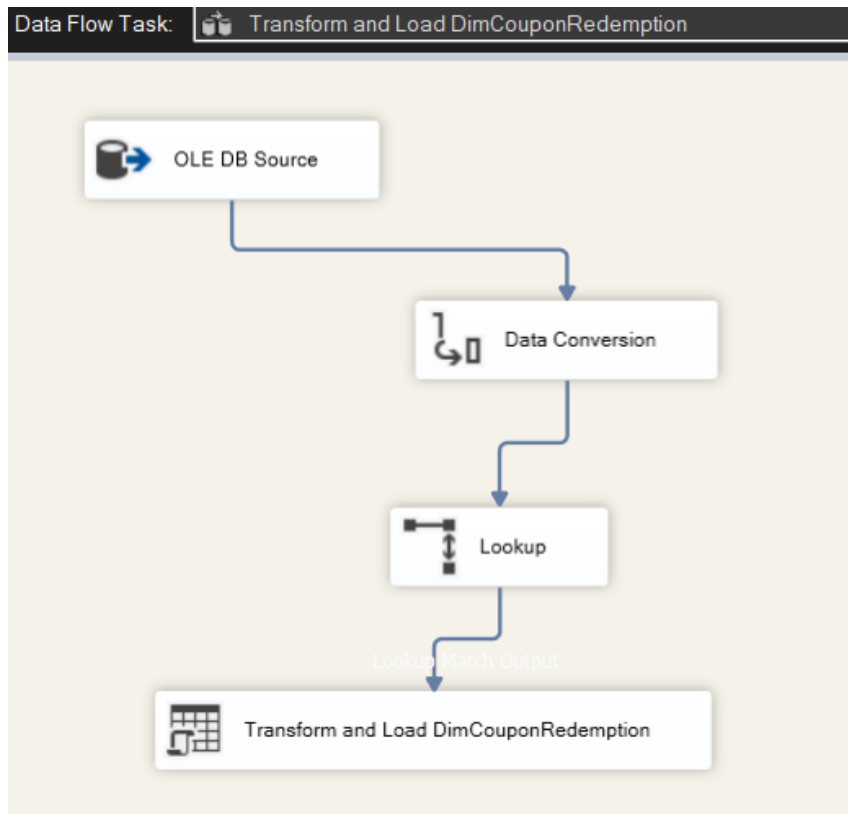






In a similar way campaign and Coupon data were loaded into the Coupon\_DW.





 Lookup Transformation Editor

This transform enables the performance of simple equi-joins between the input and a reference data set.

General  
Connection  
**Columns**  
Advanced  
Error Output

**Available Input Columns**

| Name              |
|-------------------|
| campaign_id       |
| coupon_id         |
| redemption_status |
| campaign_id_conv  |

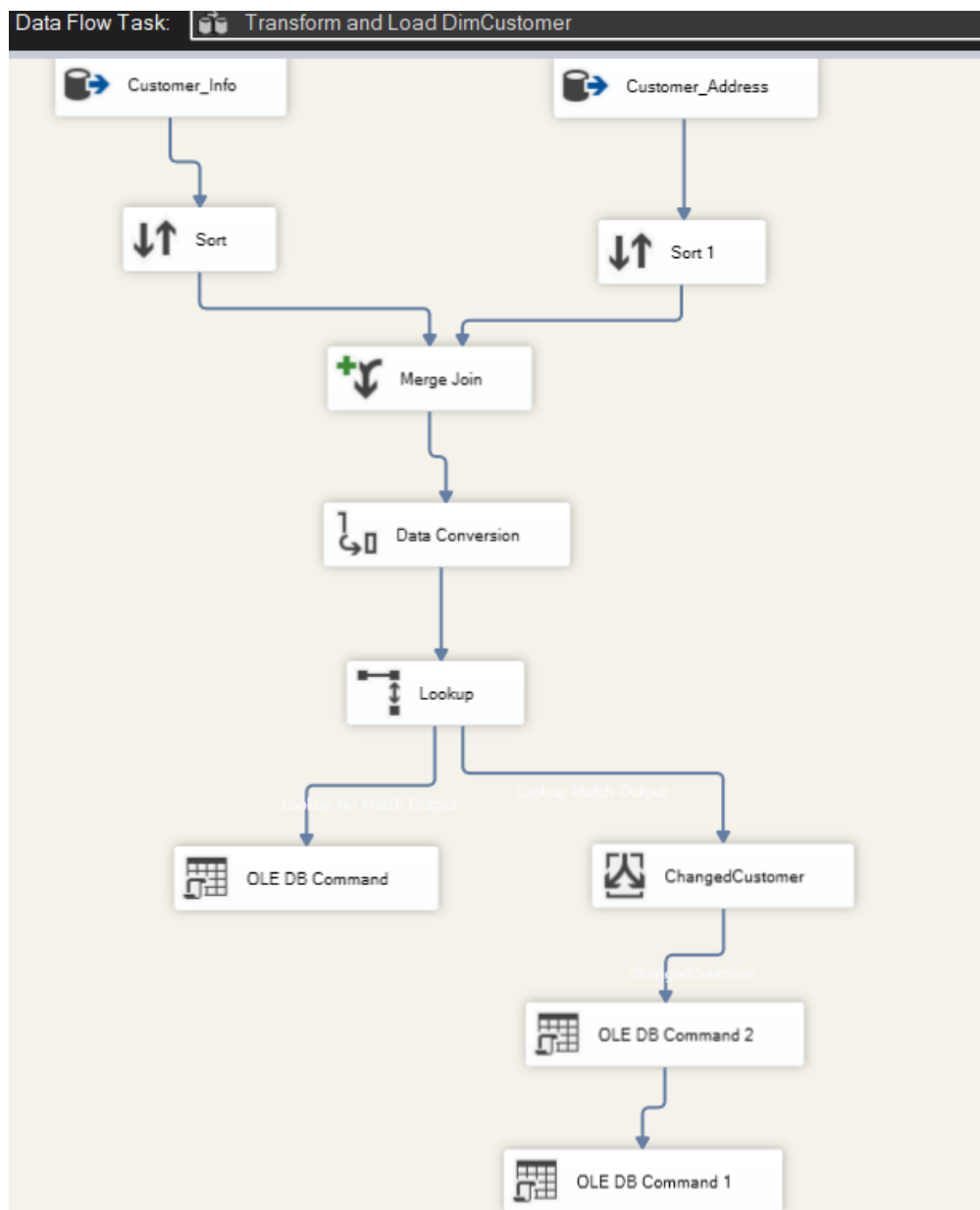
**Available Lookup Columns**

| <input checked="" type="checkbox"/> | Name                | Index |
|-------------------------------------|---------------------|-------|
| <input checked="" type="checkbox"/> | CampaignSK          |       |
| <input type="checkbox"/>            | CampaignAlternateID |       |
| <input type="checkbox"/>            | CampaignType        |       |
| <input type="checkbox"/>            | StartDate           |       |
| <input type="checkbox"/>            | EndDate             |       |
| <input type="checkbox"/>            | InsertDate          |       |

| Lookup Column | Lookup Operation    | Output Alias |
|---------------|---------------------|--------------|
| CampaignSK    | <add as new column> | CampaignSK   |

## 6.2.2. Slowly Changing Dimension Table

- Customer Dimension is assumed to be the slowly changing dimension of the data warehouse where history management of such data are needed in order to implement specific marketing strategies thus maintaining better customer satisfaction



- Initially, customer information and customer address details were available as two separate datasets in the staging area. These datasets were first sorted using the CustomerID to prepare them for merging. A Merge Join transformation was then used to combine both datasets into a single unified customer record.

**Merge Join Transformation Editor**

Configure the properties used to join two sources of sorted data. Select the join type and then specify the columns to be used as the join key. Join keys must be used in the order specified by the sort-key position of the column.

Join type: Left outer join Swap Inputs

Sort

|                                     | Name           | Ord... | Join...                             |
|-------------------------------------|----------------|--------|-------------------------------------|
| <input checked="" type="checkbox"/> | customer_id    | 1      | <input checked="" type="checkbox"/> |
| <input checked="" type="checkbox"/> | age_range      | 0      | <input type="checkbox"/>            |
| <input checked="" type="checkbox"/> | marital_status | 0      | <input type="checkbox"/>            |
| <input checked="" type="checkbox"/> | family_size    | 0      | <input type="checkbox"/>            |

Sort 1

|                                     | Name         | Ord... | Join...                             |
|-------------------------------------|--------------|--------|-------------------------------------|
| <input type="checkbox"/>            | customer_id  | 1      | <input checked="" type="checkbox"/> |
| <input checked="" type="checkbox"/> | address_line | 0      | <input type="checkbox"/>            |
| <input checked="" type="checkbox"/> | city         | 0      | <input type="checkbox"/>            |

| Input  | Input Column   | Output Alias   |
|--------|----------------|----------------|
| Sort   | customer_id    | customer_id    |
| Sort   | age_range      | age_range      |
| Sort   | marital_status | marital_status |
| Sort   | family_size    | family_size    |
| Sort   | income_bracket | income_bracket |
| Sort 1 | address_line   | address_line   |
| Sort 1 | city           | city           |
| Sort 1 | contact_number | contact_number |

- After merging, a Data Conversion transformation was applied to ensure that all data types were compatible with the data warehouse schema, particularly to resolve Unicode and non-Unicode mismatches.

Data Conversion Transformation Editor

Configure the properties used to convert the data type of an input column to a different data type. Depending on the data type to which the column is converted, set the length, precision, scale, and code page of the column.

Available Input Columns

☒ Name  
☒ customer\_id  
☒ age\_range  
☒ marital\_status  
☒ family\_size  
☒ income\_bracket  
☒ address\_line  
...

| Input Column   | Output Alias        | Data Type                 | Length | Precision | Scale | Code Page |
|----------------|---------------------|---------------------------|--------|-----------|-------|-----------|
| customer_id    | customer_id_conv    | string [DT_STR]           | 50     |           |       | 1252 (ANS |
| family_size    | family_size_conv    | four-byte signed integ... |        |           |       |           |
| income_bracket | income_bracket_conv | four-byte signed integ... |        |           |       |           |
| address_line   | address_line_conv   | string [DT_STR]           | 255    |           |       | 1252 (ANS |
| age_range      | age_range_conv      | string [DT_STR]           | 50     |           |       | 1252 (ANS |
| contact_number | contact_number_conv | string [DT_STR]           | 50     |           |       | 1252 (ANS |
| marital_status | marital_status_conv | string [DT_STR]           | 50     |           |       | 1252 (ANS |
| city           | city_conv           | string [DT_STR]           | 100    |           |       | 1252 (ANS |
|                |                     |                           |        |           |       |           |

- A Lookup transformation was then used to compare incoming customer records with existing records in the DimCustomer table using the business key (CustomerAlternateID). This step helps to identify whether a customer already exists in the dimension table or is a new entry.

Lookup Transformation Editor

This transform enables the performance of simple equi-joins between the input and a reference data set.

General  
Connection  
**Columns**  
Advanced  
Error Output

Available Input Co...

| Name             |
|------------------|
| customer_id      |
| age_range        |
| marital_status   |
| family_size      |
| income_bracket   |
| address_line     |
| city             |
| contact_number   |
| customer_id_c... |

Available Lookup Columns

|                                     | Name                | Index |
|-------------------------------------|---------------------|-------|
| <input type="checkbox"/>            | CustomerAlternateID |       |
| <input checked="" type="checkbox"/> | AgeRange            |       |
| <input checked="" type="checkbox"/> | MaritalStatus       |       |
| <input checked="" type="checkbox"/> | FamilySize          |       |
| <input checked="" type="checkbox"/> | IncomeBracket       |       |
| <input checked="" type="checkbox"/> | AddressLine         |       |
| <input checked="" type="checkbox"/> | City                |       |
| <input checked="" type="checkbox"/> | ContactNumber       |       |

| Lookup Column | Lookup Operation    | Output Alias  |
|---------------|---------------------|---------------|
| AgeRange      | <add as new column> | AgeRange      |
| MaritalStatus | <add as new column> | MaritalStatus |
| FamilySize    | <add as new column> | FamilySize    |
| IncomeBracket | <add as new column> | IncomeBracket |
| AddressLine   | <add as new column> | AddressLine   |
| City          | <add as new column> | City          |
| ContactNumber | <add as new column> | ContactNumber |
| CustomerSK    | <add as new column> | CustomerSK    |

- Once the lookup process was completed, a Conditional Split transformation was used to detect changes in customer attributes. Attributes such as age\_range, marital\_status, family\_size, income\_bracket, address\_line, city, and contact\_number were compared with existing values. If any of these attributes were different, the record was identified as a changed customer.

Conditional Split Transformation Editor

Specify the conditions used to direct input rows to specific outputs. If an input row matches no condition, the row is directed to a default output.

+ Variables and Parameters  
+ Columns

+ Mathematical Functions  
+ String Functions  
+ Date/Time Functions  
+ NULL Functions  
+ Type Casts  
+ Operators

Description:

| Order | Output Name     | Condition                                               |
|-------|-----------------|---------------------------------------------------------|
| 1     | ChangedCustomer | age_range != AgeRange    marital_status != MaritalSt... |

Default output name: Nochange

- Since the customer dimension is implemented using **Slowly Changing Dimension (SCD) Type 2** methodology, historical changes are preserved. Instead of updating existing records, a new record is inserted whenever a change is detected.
- For changed records, the existing record in the DimCustomer table is first updated using an OLE DB Command to mark it as inactive (for example, by updating the IsCurrent flag and setting the EndDate). Then, a new record is inserted with the updated attribute values, along with a new surrogate key and current StartDate.
- For new customers (no match from the lookup), records are directly inserted into the DimCustomer table.
- StartDate and EndDate columns are maintained in the dimension table to manage historical tracking of customer data. This allows the system to maintain multiple versions of a customer record over time.
- In this implementation, only SCD Type 2 is used. All selected attributes are treated as historical attributes, ensuring that any change in customer information results in the creation of a new record rather than overwriting existing data. This design supports accurate historical analysis and improves the quality of business intelligence reporting.



### 6.2.3. Fact Table

The final step of data extraction and loading to data warehouse is the process of loading data to Transaction Fact table. To complete it lookup components were used to obtain the key columns in the Transaction fact table.

Transaction fact table consists of four look up components referring DimItem , DimCustomer, DimCoupon\_Redem and DimDate, respectively. The data flow for Transaction Fact table loading is as given below.



## Steps in Transaction Fact loading

1. Customer Transaction details from Stg\_ItemTransaction are extracted from Staging table using an OLE DB Source component.
2. A lookup component is added to extract the Date SK from Dimdate by mapping Transaction date in Stg\_ItemTransaction table with Date column in order to look up to the Surrogate key of DimDate.
3. A second look up component is added to extract the SK from DimCustomer by mapping Customer\_ID in Stg\_ItemTransaction table with CustomerAlternateID in DimCustomer in order to look up to the Surrogate key of DimCustomer.
4. Continuing from step 3 another look up component is added to extract the Item SK from DimItem by mapping Item\_ID in Stg\_ItemTransaction table with ItemAlternateID in DimItem in order to look up to the Surrogate key of DimItem.
5. Then Derived Column components are added to incorporate Insert date / Modified date, ExpectedCustomerExpense, CouponUsedExpense, TotalDiscountedExpense into Transaction fact.
6. Finally, the combined data is loaded to the Fact\_transaction in the datawarehouse with the usage of OLE DB Destination component.

## Below attached are some snapshots of implemented Stored procedures

```

182
183 CREATE OR ALTER PROCEDURE dbo.UpdateDimProductBrand
184     @BrandID NVARCHAR(50),
185     @BrandType NVARCHAR(100),
186     @CategoryKey INT
187 AS
188 BEGIN
189     IF NOT EXISTS (
190         SELECT 1
191         FROM dbo.DimProductBrand
192         WHERE BrandAlternateID = @BrandID
193     )
194     BEGIN
195         INSERT INTO dbo.DimProductBrand
196         (
197             BrandAlternateID,
198             BrandType,
199             CategoryIDKey,
200             InsertDate,
201             ModifiedDate
202         )
203         VALUES
204         (
205             @BrandID,
206             @BrandType,
207             @CategoryKey,
208             GETDATE(),
209             GETDATE()
210         );
211     END
212 ELSE
213     BEGIN
214         UPDATE dbo.DimProductBrand
215         SET
216             BrandType = @BrandType,
217             CategoryIDKey = @CategoryKey,
218             ModifiedDate = GETDATE()
219         WHERE BrandAlternateID = @BrandID;
220     END
221 END;
222 GO
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

### 6.2.3.1. Accumulating Fact Table (FactTransaction)

The FactTransaction table was extended to support an accumulating fact table design in order to track the lifecycle of each transaction.

Initially, three additional columns were added to the FactTransaction table to capture the creation time, completion time, and processing duration of each transaction. These columns are:

- accm\_txn\_create\_time – represents the timestamp when the transaction is created.
- accm\_txn\_complete\_time – represents the timestamp when the transaction is completed.
- txn\_process\_time\_hours – represents the total time taken to complete the transaction in hours.

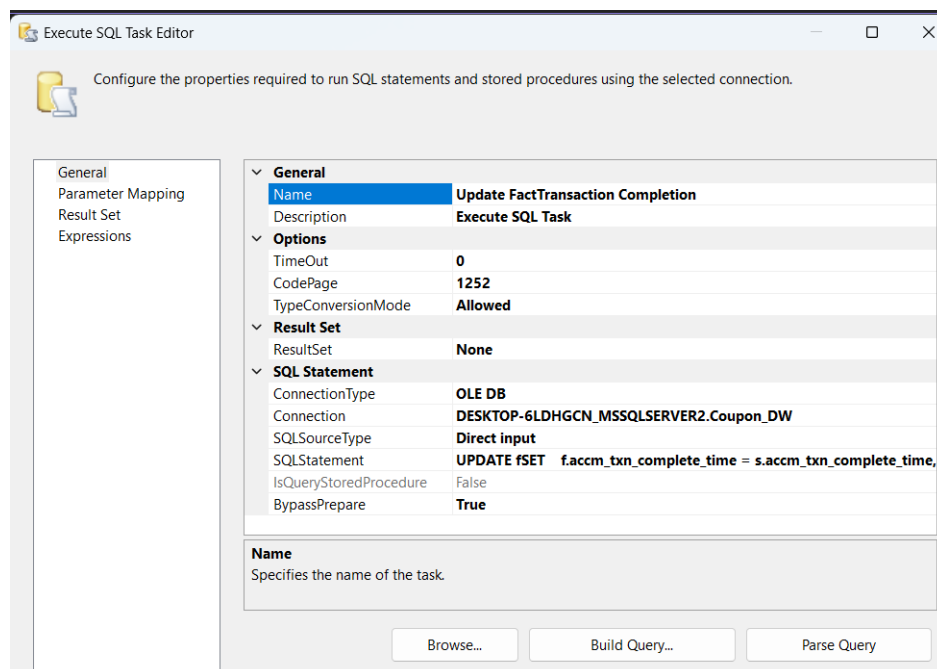
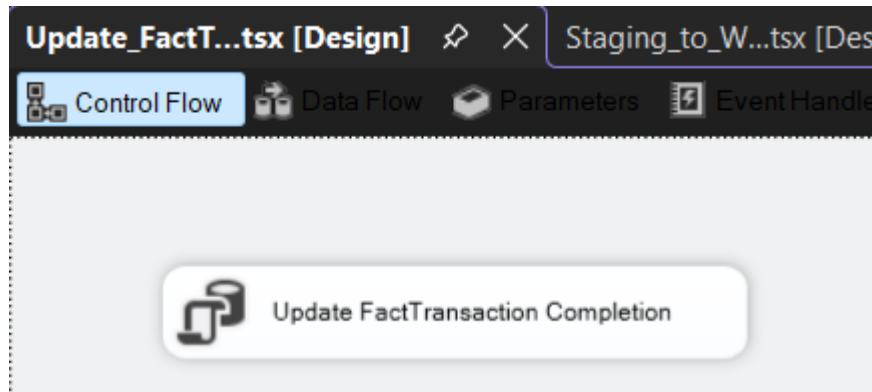
```
ALTER TABLE dbo.FactTransaction
ADD
    accm_txn_create_time DATETIME NULL,
    accm_txn_complete_time DATETIME NULL,
    txn_process_time_hours DECIMAL(18,2) NULL;
GO
```

During the initial ETL load, the accm\_txn\_create\_time was assigned using the current system date. This was implemented using a Derived Column transformation in the SSIS data flow, ensuring that every transaction record captures the exact load time.

To support the accumulating fact table requirement, a separate dataset was prepared containing transaction completion details. This dataset includes the transaction identifier (txn\_id) and the corresponding accm\_txn\_complete\_time. This dataset was stored in the staging area as a table.

```
267 CREATE TABLE dbo.Stg_FactTransactionCompletion (
268     txn_id NVARCHAR(50),
269     accm_txn_complete_time DATETIME
270 );
271 GO
272
273 TRUNCATE TABLE dbo.Stg_FactTransactionCompletion;
274 GO
275
276 INSERT INTO Coupon_Staging.dbo.Stg_FactTransactionCompletion
277 (
278     txn_id,
279     accm_txn_complete_time
280 )
281 SELECT TOP (50000)
282     TransactionAlternateID,
283     DATEADD(DAY, (ABS(CHECKSUM(NEWID())) % 5) + 1, accm_txn_create_time)
284 FROM Coupon_DW.dbo.FactTransaction
285 ORDER BY NEWID();
286 GO
287
288
```

A separate SSIS package was developed to update the FactTransaction table using this completion dataset. Instead of row-by-row updates, a set-based approach was used through an Execute SQL Task to improve performance and efficiency.



The update process joins the FactTransaction table with the staging completion dataset using the transaction identifier. For matching records, the accm\_txn\_complete\_time is updated accordingly.

Additionally, the txn\_process\_time\_hours is calculated as the difference between the completion time and the creation time. This value represents the total processing duration of each transaction and is updated during the same ETL process.

This accumulating fact table design enables tracking of transaction progress over time and provides valuable insights into processing delays and operational performance.